Proxy Pattern

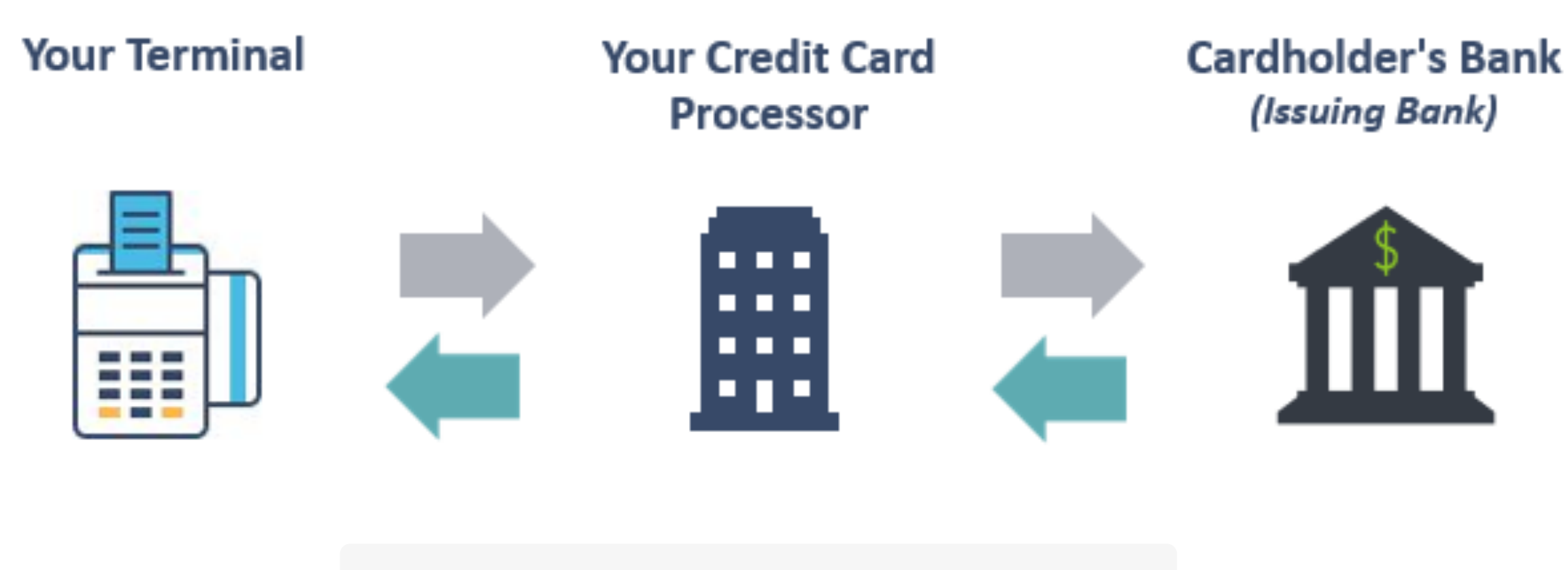
The Proxy Pattern is a design principle where a substitute, or 'proxy,' controls access to another object. It's used to add extra functionality, such as managing complex operations, controlling resource access, or handling network communication.

Consider a situation in which you have a highly valuable or complex object. You wouldn't want everyone to have direct access to it, right? Perhaps you just want to add some extra logic, like access control or lazy initialization, or it's too sensitive or heavy. This is where the Proxy Pattern becomes useful.

The Proxy Pattern involves creating a 'proxy' or stand-in for another object. This proxy intercepts all calls to the original object, allowing it to perform operations before or after forwarding the request. It's similar to having an intermediary or gatekeeper in charge of granting access to the actual object.

Real-World Example

Imagine you have a credit card - it's a proxy for your bank account. Your card does not immediately process the payment when you swipe it. It forwards the request to the bank, which handles all the processing. Your card is just the face, the interface you interact with.



Proxy Pattern - Credit Card Payment Process

In this illustration, the credit card serves as a proxy for the bank account. The credit card is shown forwarding payment requests to the bank, representing the proxy's role in controlling and managing access to the actual object, in this case, the bank account.

Structure of Proxy Pattern

The Proxy Pattern typically involves the following key components:

1. **Subject Interface**:

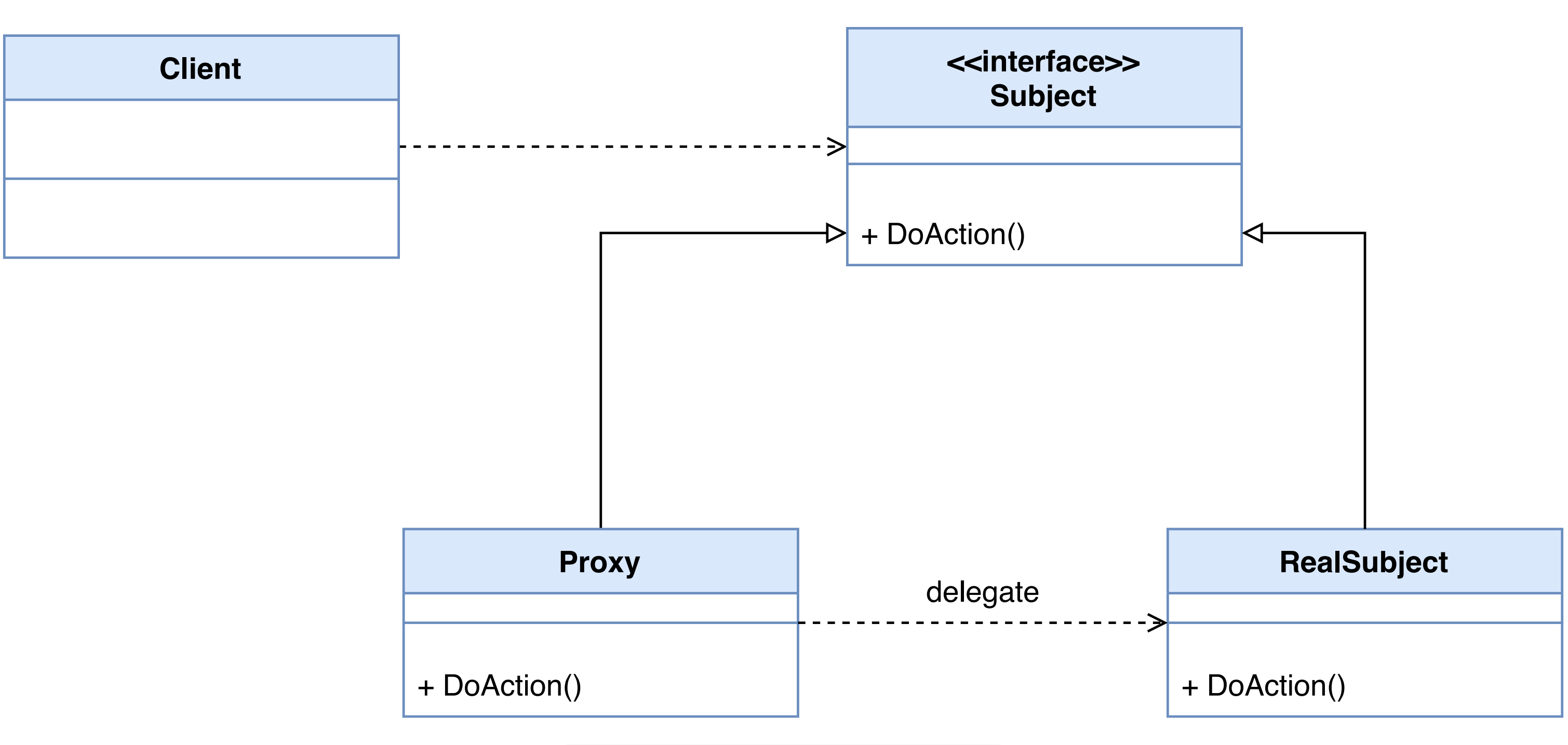
   The common methods for the actual object and the proxy are defined by this interface, also known as an abstract class. It guarantees that the real object can be replaced with the proxy.

2. **Real Subject** (**Real Object**):

   The real thing that the stand-in represents. This class defines the operations the proxy is expected to carry out and implements the Subject interface.

3. **A proxy**

   A class that serves as a proxy for the Real Subject and additionally implements the Subject interface. It keeps track of the Real Subject, manages access to it, and might even be responsible for creating and removing it. In addition, the proxy frequently handles resource-intensive operations, logs, access control, and lazy initialization.



Proxy Pattern - Class Diagram

Implementation of Proxy Pattern

Consider that you are working on a database-driven application where some queries are complex and take a long time. To maximize performance, you can implement a query caching mechanism using the Proxy Pattern. The proxy will intercept database queries and determine whether thea result is already cached before directly executing all queries on the database. It returns the cached result if it is already in the cache; if not, it runs the query and saves the result in the cache.

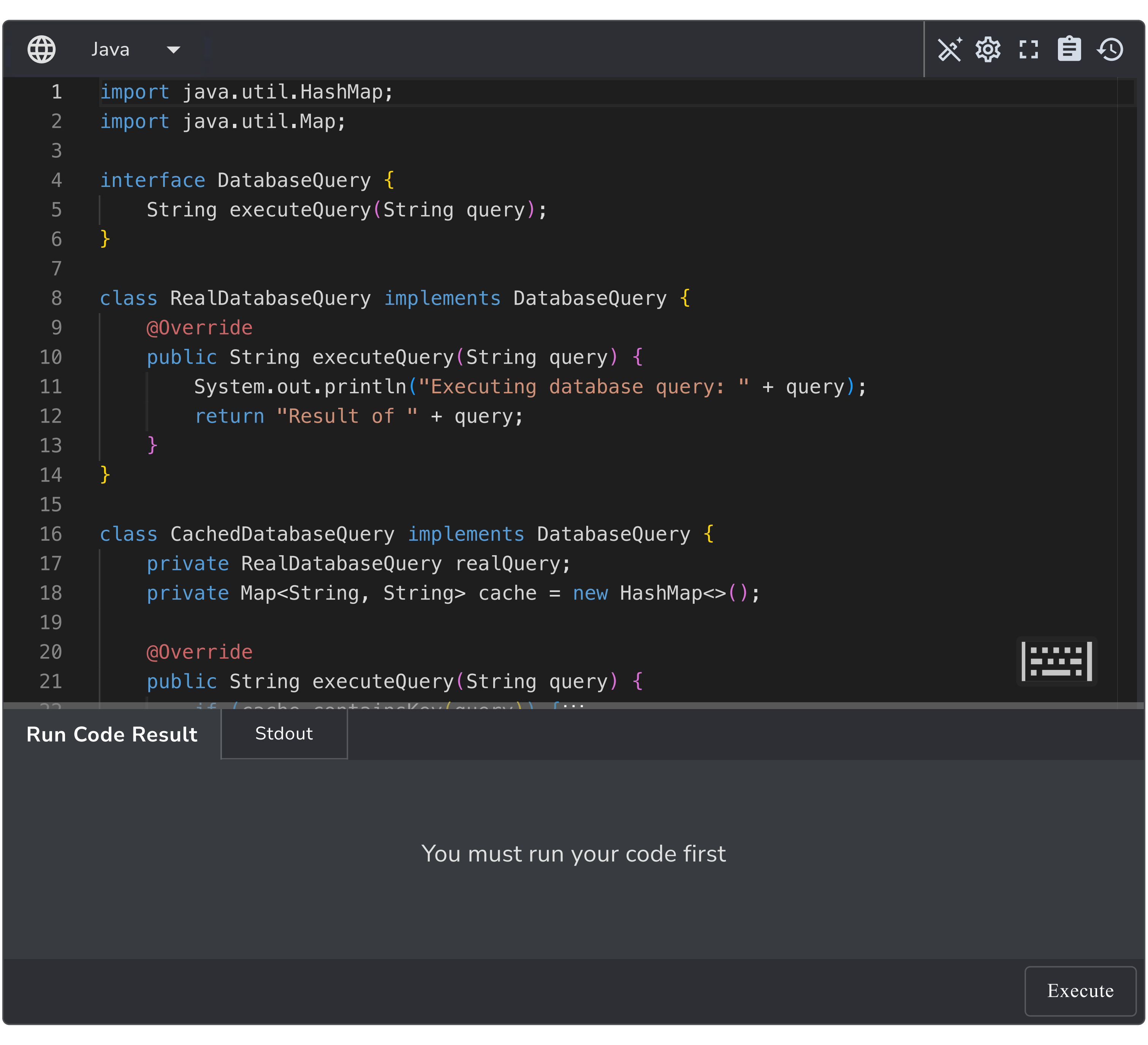
Let's have a look at the pseudocode for this example scenario:

```
// Interface for both RealDatabaseQuery and CachedDatabaseQuery
INTERFACE DatabaseQuery
    METHOD executeQuery(query: STRING): STRING

// RealDatabaseQuery class that executes a database query
CLASS RealDatabaseQuery IMPLEMENTS DatabaseQuery
    METHOD executeQuery(query: STRING): STRING
        // Code to connect and execute query on the database
        PRINT "Executing database query: " + query
        RETURN "Result of " + query

// CachedDatabaseQuery class that acts as a proxy for RealDatabaseQuery
CLASS CachedDatabaseQuery IMPLEMENTS DatabaseQuery
    PRIVATE realQuery: RealDatabaseQuery
    PRIVATE cache: MAP[STRING, STRING]

    METHOD executeQuery(query: STRING): STRING
        IF cache.contains(query)
            PRINT "Returning cached result for query: " + query
            RETURN cache.get(query)
        ELSE
            IF realQuery IS NULL
                realQuery = new RealDatabaseQuery()
            result = realQuery.executeQuery(query)
            cache.put(query, result)
            RETURN result

// Client code that uses the CachedDatabaseQuery
CLASS Application
    METHOD main()
        query: DatabaseQuery = new CachedDatabaseQuery()

        // The result will be cached for future requests
        PRINT query.executeQuery("SELECT * FROM users") // Executes and caches
        PRINT query.executeQuery("SELECT * FROM users") // Returns cached result
```

- **DatabaseQuery Interface**:
  - Interface that has a method `executeQuery()`. Both the actual and proxy database query will implement this interface.
- **RealDatabaseQuery Class**:
  - The actual database query executor. It directly interacts with the database to execute queries.
- **CachedDatabaseQuery Class**:
  - The proxy for `RealDatabaseQuery`. It maintains a cache of query results. When a query is executed, it first checks the cache. If the result is cached, it returns the cached result. Otherwise, it executes the query using `RealDatabaseQuery`, caches the result, and then returns it.
- **Application** (**Client**):
  - Uses `CachedDatabaseQuery` to handle database queries. The caching mechanism is transparent to the client, interacting with the `DatabaseQuery` interface.

This example demonstrates optimizing database operations using the Proxy Pattern—a critical component in applications where resource management and performance are critical. The pattern makes efficient use of resources by avoiding redundant database queries.

Implementation



```java
import java.util.HashMap;
import java.util.Map;

interface DatabaseQuery {
    String executeQuery(String query);
}

class RealDatabaseQuery implements DatabaseQuery {
    @Override
    public String executeQuery(String query) {
        System.out.println("Executing database query: " + query);
        return "Result of " + query;
    }
}

class CachedDatabaseQuery implements DatabaseQuery {
    private RealDatabaseQuery realQuery;
    private Map<String, String> cache = new HashMap<>();

    @Override
    public String executeQuery(String query) {
```

```
You must run your code first
```

Application of Proxy Pattern

Proxy Pattern is a versatile design pattern. There are many applicability scenarios where it can be helpful. Let's discuss some of these here:

- **Lazy Initialization**

  Proxy Pattern helps you when you have an object that is expensive to initialize. The proxy can delay its initialization till the time it is really needed. For example, any document editor can use this pattern to delay the loading process of some heavy documents or images.

- **Restricted Access**

  If you want to restrict the access of some object, you can create a proxy as a gatekeeper for that object. This is useful when you need to secure some sensitive data in your system.

- **Logging Requests**

  It can be used to log the number of requests made to some object. This feature is particularly useful when you need to log the number of requests made to a particular data source, such as a database.

- **Caching Resources**

  Proxies can also be used to cache the data that are resource-intensive. For example, for some network requests or some complex computations, proxies can cache the results for the first time and then use the cached results for subsequent requests.

In essence, the proxy pattern is helpful when you need to add some additional or extra layer of control between the client and your object. Proxy patterns can be helpful in adding some functionality to any object without changing its interface or client code.

Pros and Cons

The Proxy Pattern, like any design pattern, comes with its own set of advantages and disadvantages. Here's a breakdown of its pros and cons:

Pros	Cons
Controlled Access: It increases the security of objects by managing access to real objects.	**Increased Complexity**: Adding new layers to the system can eventually increase the complexity.
Reduced Complexity: It helps simplify the interaction among complex objects.	**Performance Overhead**: This can introduce latency, especially in remote proxies.
Enhanced Performance: Performance can be enhanced using the proxy pattern for caching and lazy loading of resource-intensive objects.	**Design Complexity**: It requires careful software design when you need a sync between real and proxy objects.
**Isolation from Clients: **This shields clients from the complexity of the real object.	**Transparency Issues**: The behavior of a proxy may not match that of the real object, which can result in unexpected outcomes.
Additional Functionality: Can add functionalities like logging, monitoring, and constraint enforcement.	

The Proxy Pattern uses a 'proxy'—a stand-in for another object. This proxy manages access to the original object, adding extra actions like security checks or logging whenever the object is used. It can delay the creation of expensive objects until they're needed, improving performance. Proxy can also act as an intermediary to handle complex network requests or to protect the object from unauthorized access.

← Previous
Flyweight Pattern
Next →
Summary
☐ Mark as Completed